

Задание №4.

Пользовательские элементы управления

Цель задания:

- Ознакомиться с механизмом *DataTemplate*.
- Создать пользовательский элемент управления.
- Реализовать валидацию данных с использованием *INotifyDataErrorInfo*.

В данном задании вы подробнее изучите возможности по созданию пользовательских элементов управления и кастомизации пользовательского интерфейса.

Item Template и Data Template

Wpf имеет широкие возможности по верстке элементов. Например, он позволяет переопределить шаблон оформления элементов списка `ListBox` и других элементов управления коллекциями. Шаблон верстки элемента списка хранится в свойстве `ListBox.ItemTemplate` и имеет тип данных `DataTemplate`. `ItemTemplate` можно определить непосредственно в xaml:

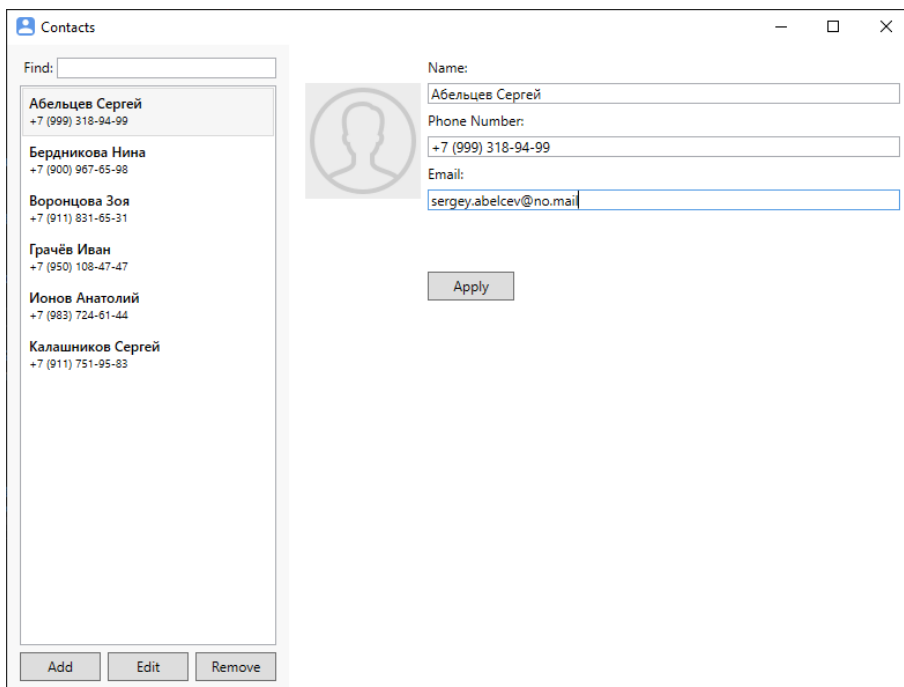
```
<ListBox
  Grid.Column="0"
  Margin="3"
  Grid.Row="1"
  Grid.ColumnSpan="3"
  ItemsSource="{Binding Contacts}">
  <ListBox.ItemTemplate>
    <DataTemplate>
      <StackPanel Margin="3">
        <TextBlock
          FontSize="12"
          FontWeight="SemiBold"
          Text="{Binding Name}"/>
        <TextBlock
```

```

        FontSize="10"
        Text="{Binding PhoneNumber}"/>
    </StackPanel>
</DataTemplate>
</ListBox.ItemTemplate>
</ListBox>

```

При использовании представленного выше шаблона, ListBox будет отображаться следующим образом:



Важное преимущество ItemTemplate, что мы можем использовать сколько угодно сложную верстку с любыми элементами управления. Например, мы можем добавить в шаблон элемента два текстовых поля с разными стилями, которые привязаны к разным свойствам ListBoxItem, как это показано в коде выше.

Обратите внимание, что в качестве привязки для текста мы указываем названия свойств Name или PhoneNumber:

```

<TextBlock Text="{Binding Name}"/>

```

```
<TextBlock Text="{Binding PhoneNumber}"/>
```

В классе MainVM нет свойств с такими названиями, но мы пытаемся привязаться и не к MainVM, а свойствам тех объектов, которые хранятся в ItemsSource нашего ListBox. В нашем примере ListBox биндиться к списку Contacts класса MainVM:

```
<ListBox ItemsSource="{Binding Contacts}"/>
```

Которая является коллекцией объектов типа Contact:

```
public class MainVM: INotifyPropertyChanged
{
    ...
    public ObservableCollection<Contact> Contacts { get; set; }
}
```

Таким образом, контекстом данных для ItemTemplate является объект из списка Contacts, то есть экземпляр класса Contact. Так как у этого класса есть свойства Name и PhoneNumber, то биндинг к этим свойствам будет выдавать имена и номера телефонов при запуске приложения. При этом, xaml не важно, объекты какого класса хранятся в ItemsSource. Главное, чтобы у этих объектов были свойства Name и PhoneNumber. Если бы у объектов в списке ItemsSource не было свойств Name и PhoneNumber, то при запуске приложения у нас бы создавались элементы списка с пустыми текстовыми подписями.

С помощью ItemTemplate можно создавать достаточно сложную верстку с большим количеством элементов и сложным поведением, с размещением картинок, чекбоксов, комбобоксов и любых других элементов при необходимости.

Так как DataTemplate зачастую описываются большим количеством строк кода, их, как правило, выносят в ресурсы окна вместо объявления непосредственно внутри ListBox в xaml:

```
<Window ... Title="MainWindow">
    <Window.Resources>
        <DataTemplate x:Key="contactTemplate">
            <StackPanel Margin="3">
                <TextBlock Text="{Binding Name}"/>
                <TextBlock Text="{Binding PhoneNumber}"/>
            </StackPanel>
        </DataTemplate>
    </Window.Resources>
</Window>
```

```

</Window.Resources>
<Grid>
<ListBox
    ItemTemplate="{StaticResource contactTemplate}"/>
</Grid>
</Window>

```

Подробнее о создании ItemTemplate можно почитать [здесь](#).

Задание

- 1) Реализовать DataTemplate для списка контактов. DataTemplate должен быть размещен в ресурсах главного окна. Макет верстки:

- 2) Создать пользовательский элемент управления UserControl ContactControl, в который необходимо выделить всю правую панель главного окна за исключением кнопки Apply – она может остаться в главном окне. В качестве DataContext новому элементу управления должен присваиваться текущий выбранный объект SelectedContact (объект

класса `Contact` или `ContactVM`). Создание пользовательских элементов описано в [МакДональд, глава 18]. В проекте новый элемент управления должен находиться в подпапке `Controls`.

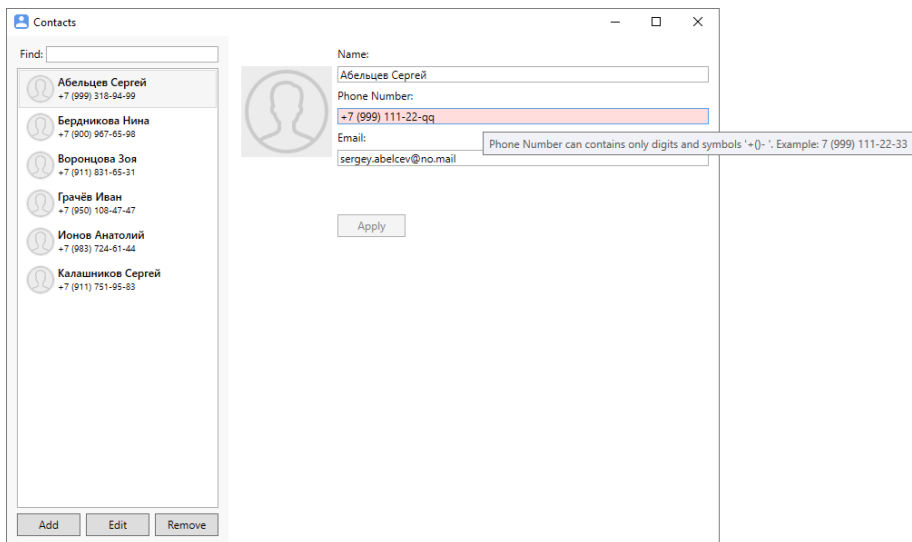
3) Реализовать валидацию вводимых пользователем данных. Должны быть реализованы следующие ограничения:

- **Name** должен быть не длиннее 100 символов
- **PhoneNumber** должен быть не длиннее 100 символов и может содержать только цифры или символы `+ - ( ) .`

Пример: `+7 (999) 111-22-33`

- **Email** должен быть не длиннее 100 символов и должен содержать символ `@`.

В случае, если пользователь неправильно ввёл значение в текстовое поле, текстовое поле должно загореться красным цветом, а во всплывающей подсказке должен отображаться текст ошибки. Текстовое поле становится обычным после того, как пользователь введёт правильное значение. Пока все поля не будут заполнены правильно, кнопка `Apply` должна быть недоступна. Макет окна с валидацией:



В Wpf есть несколько способов реализации валидации. Рекомендуем использовать реализацию валидации на основе интерфейса `IDataErrorInfo` или `INotifyDataErrorInfo`, описанные [Vice, chapter 9], или использовать сторонние библиотеки, такие как `MVVM Toolkit` или `Fluent Validation`.

- 4) Добавьте для текстового поля Phone Number в пользовательском интерфейсе преформатирование вводимого текста, запрещающий ввод любых недопустимых символов. Для этого используйте свойство `TextBox.PreviewTextInput`. Для предотвращения копирования из буфера обмена в текстовое поле недопустимых символов можно использовать событие `TextBox.DataObject.Pasting`.
- 5) Протестируйте работу приложения. Если всё работает корректно и код оформлен правильно, выполните Pull Request в ветку develop.

Литература

1. **Мак-Дональд М.**
WPF 4: Windows Presentation Foundation в .NET 4.0 с примерами на C# 2010 для профессионалов пер. с англ. – М.: ООО «И.Д. Вильямс», 2011. – 1024 с. : ил. – парал. тит. англ. (глава 18)
2. **Metanit.com. Руководство по WPF**
/ Metanit.com. – URL: <https://metanit.com/sharp/wpf/> (главы 1-7, 10)